Muntada

Product and Engineering Specification

Implementation architecture, module design, APIs, workflows, and phased backlog for Muntada

Document type	Engineering specification
Prepared for	Product and engineering planning
Primary architecture	React SPA + ASP.NET Core modular monolith + LiveKit OSS
Version	Version 1.0
Status	Draft for implementation planning

Scope baseline

Phase 1 targets the web product. Phase 2 adds React Native mobile applications on the same core business APIs. Where the client answers were intentionally left open, this document marks the item as TBD and provides a recommended default.

Document control

Field	Value
Project	Muntada
Business model	Public SaaS for individual creators, education, and enterprise tenants
Primary phase	Phase 1 web; Phase 2 React Native mobile
Selected architecture	React SPA (TypeScript) + ASP.NET Core modular monolith + LiveKit OSS
Deployment	Self-managed private cloud / hosted Kubernetes
Region target	GCC only
Status	Architecture and requirements baseline
Outstanding decision	Availability topology for 99.9% target remains TBD

1. Executive summary and architecture decision

Muntada will be built as a public, multi-tenant SaaS platform focused initially on moderated audio rooms. The selected architecture uses a React SPA implemented in TypeScript for the authenticated web product, a separate public site, an ASP.NET Core modular monolith for the control plane and business APIs, and a self-hosted LiveKit cluster for the realtime media plane. SignalR carries non-media realtime state, SQL Server stores transactional data, Redis handles caching and ephemeral presence data, RabbitMQ supports asynchronous workflows, and MinIO stores room artifacts and exports.

Decision outcome

Choose LiveKit OSS as the media engine and keep all product logic - tenancy, moderation policy, consent, billing, analytics, files, audit, and admin tooling - in the ASP.NET Core application. This preserves mobile-ready business APIs while using the most appropriate tool for low-latency realtime audio.

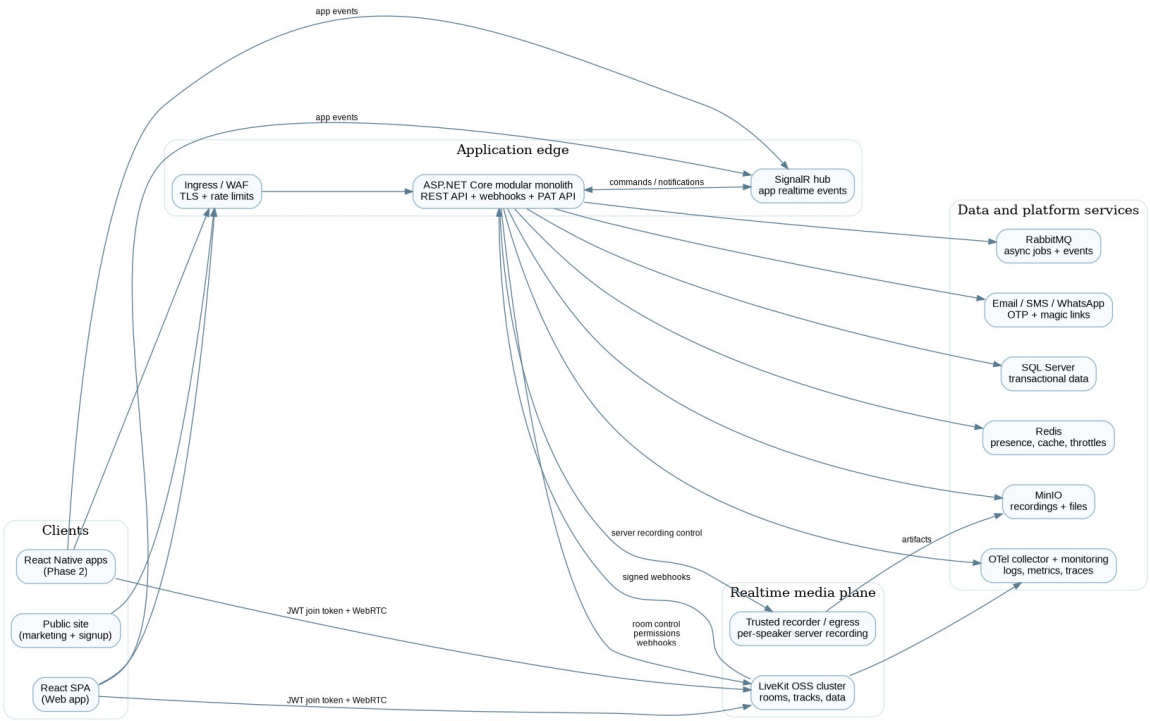

Figure 1. Engineering baseline

2. Product goals and guiding principles

- Deliver a safe, moderated, invite-only audio room experience for creators, educators, and enterprise users.
- Keep one durable business API surface for web and React Native mobile clients.
- Treat tenant configurability, auditability, and exportability as first-class features rather than afterthoughts.
- Isolate media transport concerns from business orchestration concerns.
- Prepare now for future video, screen sharing, transcription, AI summaries, and translation without forcing those features into Phase 1 UI.
- Favor explicit state machines and event contracts over hidden client logic so the platform remains predictable under scale and compliance pressure.

3. Scope by phase

Capability area	Phase 1 web	Phase 2 mobile	Future-ready now
React SPA web product	Yes	N/A	Yes

React Native iOS/Android	No	Yes	API-ready
Invite-only audio rooms	Yes	Yes	Yes
One moderator with handover	Yes	Yes	Yes
Room-wide chat + moderator DM	Yes	Yes	Yes
Local recording	Yes	Yes	Yes
Moderator server recording	Yes	Yes	Yes
Video and screen sharing	No	No	Data model + API reserved
Transcription / AI summary / translation	No	No	Reserved events and schemas
Enterprise SSO / SCIM	No	Yes	Data model ready

4. System context and bounded modules

The modular monolith is decomposed into business modules with clear command/query boundaries and integration events. Each module owns its data model slice and service layer. Shared infrastructure is limited to cross-cutting primitives such as authentication, persistence, background jobs, and telemetry.

Module	Primary responsibilities
Identity & Access	Accounts, credentials, sessions, refresh tokens, org switch, PATs, OTP challenges, guest magic links.
Tenancy & Plans	Tenant creation, branding, subdomains, plan limits, retention policies, feature toggles.
Rooms & Scheduling	Room templates, recurring series, occurrences, invites, membership, lifecycle, moderator assignment.
Realtime Orchestration	LiveKit token issuance, webhook handling, room state cache, SignalR fan-out, active recorder counter.
Moderation	Raise hand queue, speaker grants, roster states, moderator handover, lock/remove actions.
Chat & Messaging	Room-wide chat, moderator DM, delivery status, read models.
Recording & Consent	Local recording registry, server recording orchestration, manifests, download rules, consent events.

Files & Artifacts	Uploads, malware scan workflow, storage, targeted delivery, expiry, downloads.
Billing & Metering	Concurrent room meters, subscription state, OTP/storage/AI usage meters.
Reporting & Audit	Operational dashboards, tenant analytics, audit export, usage aggregates.
Admin & Support	Tenant management, live interventions, impersonation, emergency revoke.
Public API & Webhooks	PAT-protected API surface, quotas, signed webhooks, idempotency.

5. Domain model summary

The core data model must support multi-organization membership, tenant-specific policies, room artifacts, and an event-rich audit trail. The domain model should remain stable even as media capabilities expand later.

```
Tenant 1---* TenantMembership *---1 User
Tenant 1---* RoomSeries 1---* RoomOccurrence
RoomOccurrence 1---* RoomInvite
RoomOccurrence 1---* RoomParticipantState
RoomOccurrence 1---1 ModeratorAssignment
RoomOccurrence 1---* HandRaiseRequest
RoomOccurrence 1---* SpeakerGrant
RoomOccurrence 1---* ChatMessage
RoomOccurrence 1---* FileArtifact
RoomOccurrence 1---* RecordingArtifact
RecordingArtifact 1---1 RecordingManifest
Tenant 1---* AuditEvent
Tenant 1---* BillingMeterSnapshot
```

ERD note

The final implementation should create separate SQL Server schemas per module or at least enforce table naming boundaries per module. This keeps the modular monolith physically disciplined even before any future service extraction.

6. State machines

Explicit state models are essential because room lifecycle, moderator presence, and recording announcements cannot be inferred safely from UI state alone.

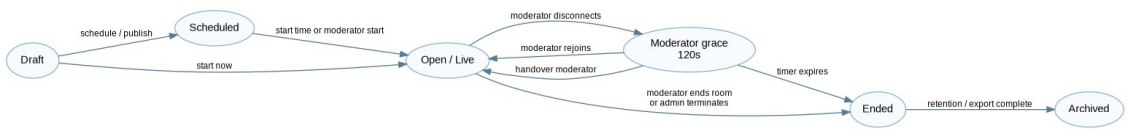

Figure 2. Room lifecycle state machine

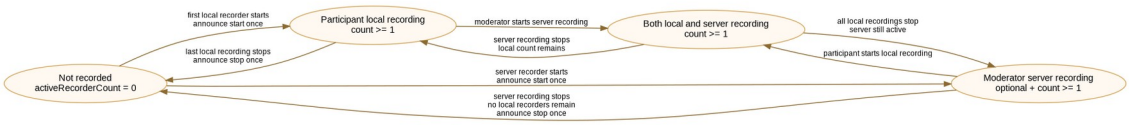

Figure 3. Shared recording-state model

State object	Allowed states	Notes
RoomOccurrence.Status	Draft, Scheduled, Live, Grace, Ended, Archived	Grace begins only when moderator disconnects while room is live.
HandRaiseRequest.Status	Queued, Approved, Rejected, Cancelled, Expired	Queued order is visible to moderator.
SpeakerGrant.Status	Active, Revoked, Expired	Revocation unpublishes tracks immediately.
RecordingSession.Type	Local, Server	Both feed the active recorder counter.
RecordingSession.Status	Pending, Active, Stopping, Completed, Failed	Server recordings persist artifacts; local recordings do not.
FileArtifact.Status	Uploaded, Scanning, Ready, Rejected, Expired, Deleted	Downloads only allowed in Ready state.

7. Key workflows and sequence diagrams

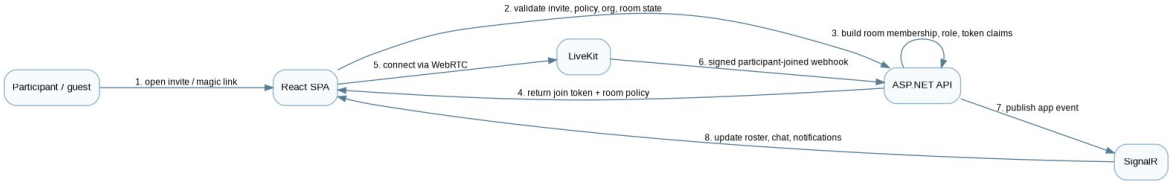

Figure 4. Join room flow

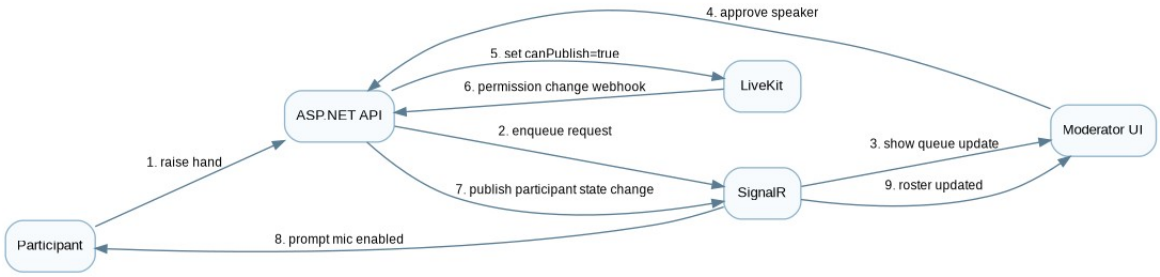

Figure 5. Raise hand and speaker approval flow

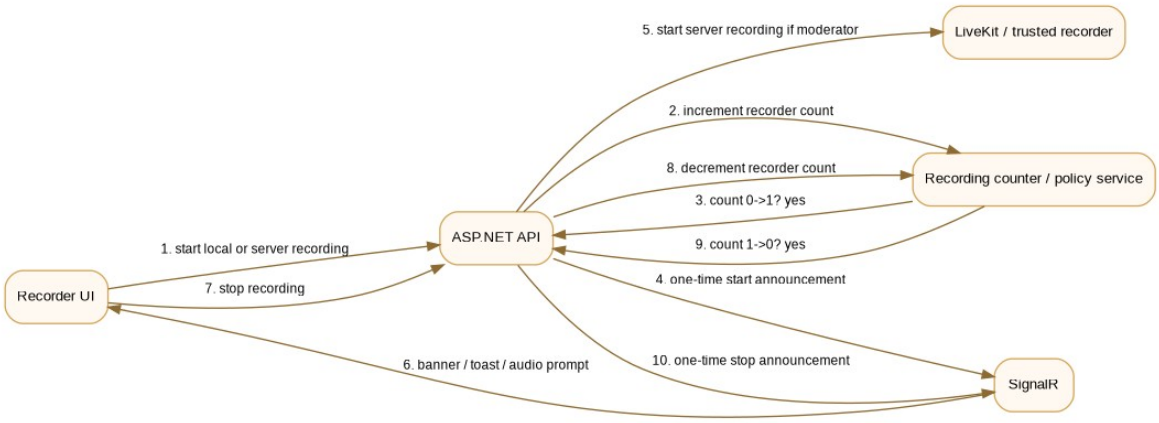

Figure 6. Recording announcement flow

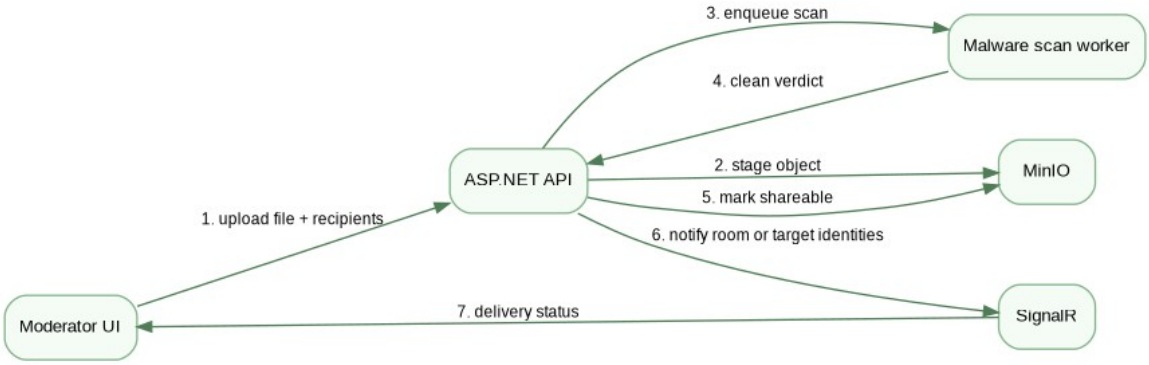

Figure 7. File upload and delivery flow

Workflow	Server authority	Client responsibility
Join room	Validate invite, tenant policy, room state, and issue join token	Acquire device permissions, connect to LiveKit, reflect UI state
Raise hand	Persist queue, publish SignalR update	Render queue state and moderator actions
Grant speaker	Update permission with LiveKit and room state cache	Enable microphone UX and reflect approval
Start local recording	Create room-level recording session record and counter transition	Capture media locally and upload nothing unless user chooses export path
Start server recording	Authorize moderator, launch trusted recorder, persist artifact metadata	Show recording state and download availability
File share	Persist artifact, scan, enforce targeted delivery, audit access	Upload file and show download affordances

8. API design principles

- Version APIs from day one using URI versioning or explicit media-type versioning; the examples below use /api/v1/.
- Separate public integration APIs from first-party app APIs logically, even if they are hosted in the same monolith.
- Use REST for commands/queries, SignalR for app event subscriptions, and LiveKit tokens/WebRTC for media transport.
- All mutating endpoints accept idempotency keys when the action may be retried from automation or unstable networks.
- All list endpoints support pagination, filtering, and stable sort order.
- Every externally visible identifier should be opaque and non-sequential.

9. API endpoint catalog (condensed)

Area	Representative endpoints
Identity	POST /api/v1/auth/login; POST /api/v1/auth/refresh; POST /api/v1/auth/otp/challenge; POST /api/v1/auth/magic-link
Organizations	GET /api/v1/orgs; POST /api/v1/orgs/switch; GET /api/v1/tenants/{id}/branding; PUT /api/v1/tenants/{id}/policies
Rooms	POST /api/v1/rooms; GET /api/v1/rooms/{id}; POST /api/v1/rooms/{id}/start; POST /api/v1/rooms/{id}/end
Invites	POST /api/v1/rooms/{id}/invites; POST /api/v1/room-invites/{token}/accept
Moderation	POST /api/v1/rooms/{id}/hands; POST /api/v1/rooms/{id}/speaker-grants; DELETE /api/v1/rooms/{id}/speaker-grants/{grantId}; POST /api/v1/rooms/{id}/lock
Realtime join	POST /api/v1/rooms/{id}/join-token
Chat	GET /api/v1/rooms/{id}/chat; POST /api/v1/rooms/{id}/chat/messages; POST /api/v1/rooms/{id}/chat/direct-messages
Recording	POST /api/v1/rooms/{id}/recordings/local/start; POST /api/v1/rooms/{id}/recordings/local/stop; POST /api/v1/rooms/{id}/recordings/server/start; POST /api/v1/rooms/{id}/recordings/server/stop

Files	POST /api/v1/rooms/{id}/files; GET /api/v1/files/{id}; POST /api/v1/files/{id}/downloads
Admin	POST /api/v1/admin/rooms/{id}/terminate; POST /api/v1/admin/rooms/{id}/participants/{pid}/remove; POST /api/v1/admin/impersonations
Reports	GET /api/v1/reports/tenant-usage; GET /api/v1/audit-events; POST /api/v1/audit-exports
Public API	GET /public/v1/rooms; POST /public/v1/rooms; GET /public/v1/usage; POST /public/v1/webhook-endpoints

API note
The local recording endpoints exist to register and audit room-level recording state, not to stream local media through the server. The actual capture occurs on the client.

10. SignalR and event contract model

SignalR is used for application-level events that should be authoritative from the control plane. LiveKit events still drive client media state, but the server publishes normalized room events so that all clients behave consistently across web and mobile.

Channel / event	Payload summary	Consumer
room.{roomId}.roster.changed	Participant presence, role, speaking, recording, quality state	Moderator and participant UIs
room.{roomId}.handraise.changed	Queue entries, approvals, rejections, sequence order	Moderator UI
room.{roomId}.announcement.changed	Announcement type, selected delivery modes, locale	All room clients
room.{roomId}.recording.counter.changed	Active count, visible recording state, moderator-only recorder detail	All room clients / moderator detail view
room.{roomId}.file.ready	Artifact id, scope, target identities, expiry, download permissions	Eligible recipients
tenant.{tenantId}.report.snapshot	Near real-time usage aggregates	Tenant admin dashboard
admin.platform.alert	Operational escalation	Platform admin console

11. Security architecture

- Authenticate end users with short-lived JWT access tokens plus refresh tokens bound to revocable sessions.
- Protect public API calls with PATs scoped to tenant and action set; store only token hashes.
- Use invite-only room membership checks before issuing any LiveKit join token.

- Represent room capabilities in join-token claims (subscribe, publish, publish data, recorder, admin controls) and refresh them after moderation changes.
- Support three effective encryption modes: transport-only, E2EE without trusted services, and E2EE with trusted recorder/AI service access.
- Persist consent and pre-join acknowledgement logs as part of recording/compliance workflows.
- Keep guest listeners strictly listen-only in Phase 1 and deny file download, chat, and local recording by policy.
- Require justification and full audit for impersonation; no silent impersonation.

Threat area	Primary control
Unauthorized room access	Invite validation + token issuance only after server-side policy checks
Privilege escalation inside room	Server-authoritative moderation endpoints and token / permission refresh
Tampering with room-state UI	SignalR events signed by session context and validated against current room membership
Malicious file uploads	Pre-download malware scan and artifact status gate
Token leakage	Short access token lifetime, hashed PATs, session revoke, room-specific LiveKit join tokens
Compliance gaps	Consent logs, recording event logs, audit export, tenant retention policy enforcement

12. Deployment topology and infrastructure

The application must be deployable to self-managed Kubernetes. Because the sponsor left the exact HA topology open, the architecture must support incremental infrastructure hardening without changing business contracts.

- Stateless workloads: React static hosting, public site, ASP.NET API pods, SignalR backplane-enabled hubs, malware-scan workers, reporting workers, webhook delivery workers.
- Stateful workloads: SQL Server, Redis, RabbitMQ, MinIO, LiveKit, optional trusted recorder / egress pool.

- Ingress/WAF tier with TLS termination, rate limiting, and bot protection.
- OpenTelemetry collector sidecars or daemonsets forwarding logs, metrics, and traces to the monitoring platform.
- GitOps- or pipeline-managed environment promotion for dev, test, staging, and production namespaces.

Kubernetes workload	Scaling note
React SPA / public site	Static or edge-cached; independent horizontal scaling.
API pods	Scale on CPU and request rate; maintain no in-memory authority for room state.
SignalR	Scale horizontally with Redis backplane or managed equivalent.
LiveKit	Scale media nodes independently; size for participant concurrency and codec mix.
Recorder / egress workers	Scale by active server recordings and artifact processing volume.
Scan workers	Scale by file upload queue depth.

13. Observability and SRE model

- Emit distributed traces for join-token issuance, room start/end, speaker grant operations, file scan workflow, and recording lifecycle.
- Publish metrics for room joins, join latency, concurrent live rooms, concurrent participants, websocket connections, webhook lag, recording counter transitions, artifact scan times, and PAT usage.
- Centralize structured audit and application logs with correlation IDs spanning API, SignalR, worker, and webhook flows.
- Create alert policies for room join failures, webhook verification failures, failed scans, recorder failures, quota breaches, and rising latency.
- Maintain dashboards for tenant and platform views; tenant dashboards expose only tenant-safe aggregated data.

14. Testing strategy

Test layer	Primary focus
Unit tests	Module-level domain rules, policy evaluation, retention enforcement, billing meter logic.
Integration tests	SQL Server + Redis + RabbitMQ + MinIO-backed flows for room creation, invites, scans, recordings, and reporting.
Contract tests	Public API, webhook schemas, SignalR payload contracts, mobile/web shared DTOs.
End-to-end tests	Browser and mobile flows for join room, raise hand, recording, file sharing, and admin intervention.
Performance tests	Join latency, concurrent room scale, websocket fan-out, webhook throughput, file scan queue performance.
Security tests	Auth/session abuse, PAT misuse, invite replay, guest escalation, file upload abuse, audit completeness.

15. Backlog and phased delivery recommendation

Release slice	Recommended outcome
Foundation sprint	Repo, CI/CD, environment model, auth shell, tenancy shell, base observability, infrastructure provisioning.
Slice 1	Tenant onboarding, user accounts, invites, room scheduling, basic room join and listen-only audio.
Slice 2	Moderator console, raise hand queue, speaker approval/revocation, room lock and removal controls.
Slice 3	Room-wide chat, moderator DM, announcement settings, roster and state synchronization via SignalR.
Slice 4	Built-in local recording registry and first-start/last-stop announcement logic.
Slice 5	Server recording, trusted recorder integration, artifact manifests, download scope selection.
Slice 6	Moderator file uploads, malware scanning, targeted delivery, expiry, audit history.
Slice 7	Billing meters, tenant analytics, audit export, PAT-based public API, signed webhooks.

Slice 8	Admin support tooling, impersonation, emergency revoke, live room interventions.
Phase 2 mobile	React Native apps consuming the same APIs and room event model.

Recommended delivery tactic
Build vertical slices that run end-to-end through API, SignalR, LiveKit integration, SQL, and audit logging. Avoid implementing dashboards or billing before the authoritative room and moderation state model is stable.

16. Risks, tradeoffs, and open decisions

- E2EE with trusted recorder access satisfies the sponsor direction but creates a trust boundary that must be documented and governed clearly.
- A modular monolith is the right starting point, but module boundaries should be enforced rigorously to avoid a tangled codebase.
- Concurrent-room billing is simpler than participant-minute billing but still requires authoritative live-room state under failure and graceful-degradation conditions.
- The chosen React Native mobile stack aligns with the web technology direction, but media-specific edge cases will still need platform testing on iOS and Android.
- The exact HA topology and operator-approved retention defaults remain open and should be resolved before production readiness review.

17. Reference appendix

- LiveKit transport overview, SDK platforms, encryption, egress, participant permissions, and byte-stream documentation.
- Internal architecture assumptions derived from the sponsor-approved requirements baseline in this conversation.
- Applicable policy targets: GDPR readiness, Saudi PDPL alignment, SOC 2-aligned controls, and GCC residency.